Experiment 3: Web API using Custom Model Class

Aim

To create an ASP.NET Core Web API using a custom model class, implement GET, POST, PUT, and DELETE operations, use the FromBody and AllowAnonymous attributes, create a custom authorization filter, and implement a custom exception filter.

Objectives

- Demonstrate creation of an action method to return a list of custom class entities.
- Create model classes (Employee, Department, and Skill).
- Use the AllowAnonymous attribute.
- Use the HttpGet action method.
- Explain the usage of the FromBody attribute.
- Read the model object from the request body.
- Demonstrate a custom authorization filter using ActionFilterAttribute.
- Demonstrate a custom exception filter using IExceptionFilter.
- Test the APIs using Swagger.

Software Requirements

- Visual Studio 2022 Community Edition
- .NET 8 SDK
- Swagger (OpenAPI)
- Postman (Optional)

Step 1: Open Existing Project

1. Open **Visual Studio 2022**.
2. Click **Open a project or solution**.
3. Open the Web API project created in Experiment 2.
4. Press **Ctrl + F5**.
5. Verify that the Swagger page opens successfully.

Step 2: Create Models Folder

1. In **Solution Explorer**, right-click the project.
2. Select **Add → New Folder**.
3. Name the folder **Models**.

Project structure becomes:

SwaggerDemo

Controllers

Models

Program.cs

appsettings.json

Step 3: Create Department Model

1. Right-click **Models**.
2. Select **Add → Class**.
3. Name the file **Department.cs**.
4. Add the Department model class.

Step 4: Create Skill Model

1. Right-click **Models**.
2. Select **Add → Class**.
3. Name the file **Skill.cs**.
4. Add the Skill model class.

Step 5: Create Employee Model

1. Right-click **Models**.
2. Select **Add → Class**.
3. Name the file **Employee.cs**.
4. Add the Employee model class with the following properties:

- Id
- Name
- Salary
- Permanent
- Department
- Skills
- DateOfBirth

Step 6: Create EmployeeController

1. Open the **Controllers** folder.
2. Open the existing **EmployeeController.cs** (or create one if it doesn't exist).
3. Add the required namespaces.
4. Decorate the controller with:
    - [ApiController]
    - [Route("api/[controller]")]
5. Inherit the controller from ControllerBase.

Step 7: Create Employee List

1. Declare a private employee list.

```
private List<Employee> employees;
```

2. Create a constructor.
3. Initialize the employee list by calling the GetStandardEmployeeList() method.

Step 8: Create GetStandardEmployeeList() Method

1. Create a private method named GetStandardEmployeeList().
2. Return a sample list of Employee objects.
3. Add Department and Skill objects inside the Employee object.

Step 9: Create GET Action Method

1. Create a GET action method.
2. Add the following attributes:
    - [HttpGet]
    - [AllowAnonymous]
    - [ProducesResponseType(StatusCodes.Status200OK)]
3. Return the employee list using Ok().

Step 10: Run and Test GET

1. Build the project (**Ctrl + Shift + B**).
2. Run the project (**Ctrl + F5**).
3. Swagger opens automatically.
4. Select **GET**.
5. Click **Try it out**.
6. Click **Execute**.
7. Verify that the employee list is displayed.
8. Verify the response status code is **200 OK**.

Step 11: Create POST Action Method

1. Create a POST action method.
2. Use the [HttpPost] attribute.
3. Use the [FromBody] attribute to receive the Employee object.
4. Add the employee to the list.
5. Return the added employee using Ok().

Step 12: Test POST

1. Run the application.
2. Open Swagger.
3. Expand the POST method.
4. Click **Try it out**.

5. Enter the Employee details in JSON format.
6. Click **Execute**.
7. Verify the response is **200 OK**.

Step 13: Create PUT Action Method

1. Create a PUT action method.
2. Use the [HttpPut("{id}")] attribute.
3. Receive the employee ID and Employee object.
4. Search for the employee.
5. Update the employee details.
6. Return the updated employee.

Step 14: Test PUT

1. Expand the PUT method in Swagger.
2. Click **Try it out**.
3. Enter the Employee ID.
4. Enter the updated Employee JSON.
5. Click **Execute**.
6. Verify the updated employee details.

Step 15: Create DELETE Action Method

1. Create a DELETE action method.
2. Use the [HttpDelete("{id}")] attribute.
3. Search the employee using the ID.
4. Remove the employee from the list.
5. Return a success message.

Step 16: Test DELETE

1. Expand the DELETE method.

2. Click **Try it out**.
3. Enter the Employee ID.
4. Click **Execute**.
5. Verify the response message **Employee Deleted Successfully**.

Step 17: Create Filters Folder

1. Right-click the project.
2. Select **Add → New Folder**.
3. Name the folder **Filters**.

Step 18: Create Custom Authorization Filter

1. Right-click **Filters**.
2. Select **Add → Class**.
3. Name the file **CustomAuthFilter.cs**.
4. Inherit the class from ActionFilterAttribute.
5. Override the OnActionExecuting() method.
6. Check whether the request contains the **Authorization** header.
7. If the Authorization header is not present, return **BadRequest** with the message:

Invalid request - No Auth token

8. If the Authorization header is present but does not contain **Bearer**, return **BadRequest** with the message:

Invalid request - Token present but Bearer unavailable

Step 19: Apply Authorization Filter

1. Open **EmployeeController**.
2. Add the following attribute above the controller:

[CustomAuthFilter]

3. Save the project.

Step 20: Test Authorization Filter

Case 1

Do not send the Authorization header.

Expected Result:

Invalid request - No Auth token

Case 2

Send an Authorization header without **Bearer**.

Expected Result:

Invalid request - Token present but Bearer unavailable

Case 3

Send

Authorization : Bearer abc123

Expected Result:

The API executes successfully.

Step 21: Install Required NuGet Package

1. Right-click the project.
2. Select **Manage NuGet Packages**.
3. Search for

Microsoft.AspNetCore.Mvc.WebApiCompatShim

4. Install the package.

Step 22: Create Custom Exception Filter

1. Right-click **Filters**.
2. Select **Add → Class**.
3. Name the file **CustomExceptionFilter.cs**.
4. Implement the IExceptionFilter interface.
5. Override the OnException() method.
6. Capture the exception details.
7. Write the exception details to a text file.

8. Set the exception result.

Step 23: Apply Exception Filter

Register the exception filter in the application and apply it to the EmployeeController.

Step 24: Throw an Exception

Modify the GET action method.

Throw an exception intentionally.

Example:

throw new Exception("Sample Exception");

Step 25: Test Exception Filter

1. Run the application.
2. Open Swagger.
3. Execute the GET method.
4. Verify that:
    - HTTP Status Code is **500 Internal Server Error**.
    - Exception details are written to the log file.

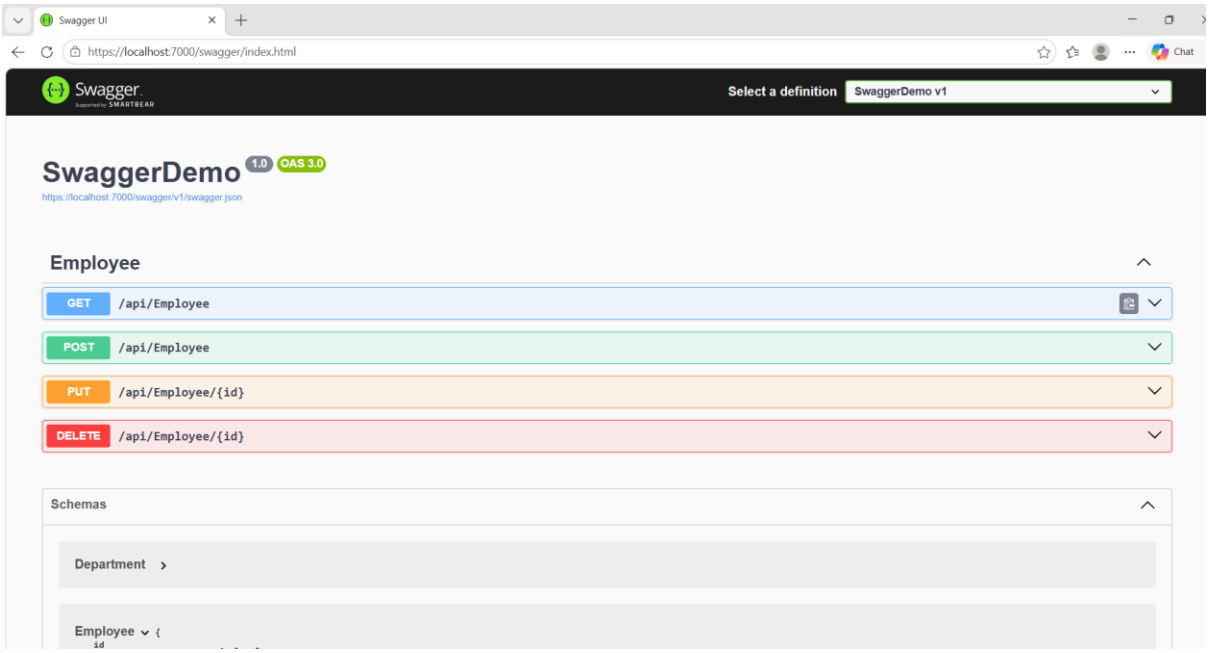
Step 26: Build and Execute

1. Build the project.
2. Run the project.
3. Test all APIs using Swagger.
4. Verify GET, POST, PUT, DELETE, Authorization Filter, and Exception Filter.

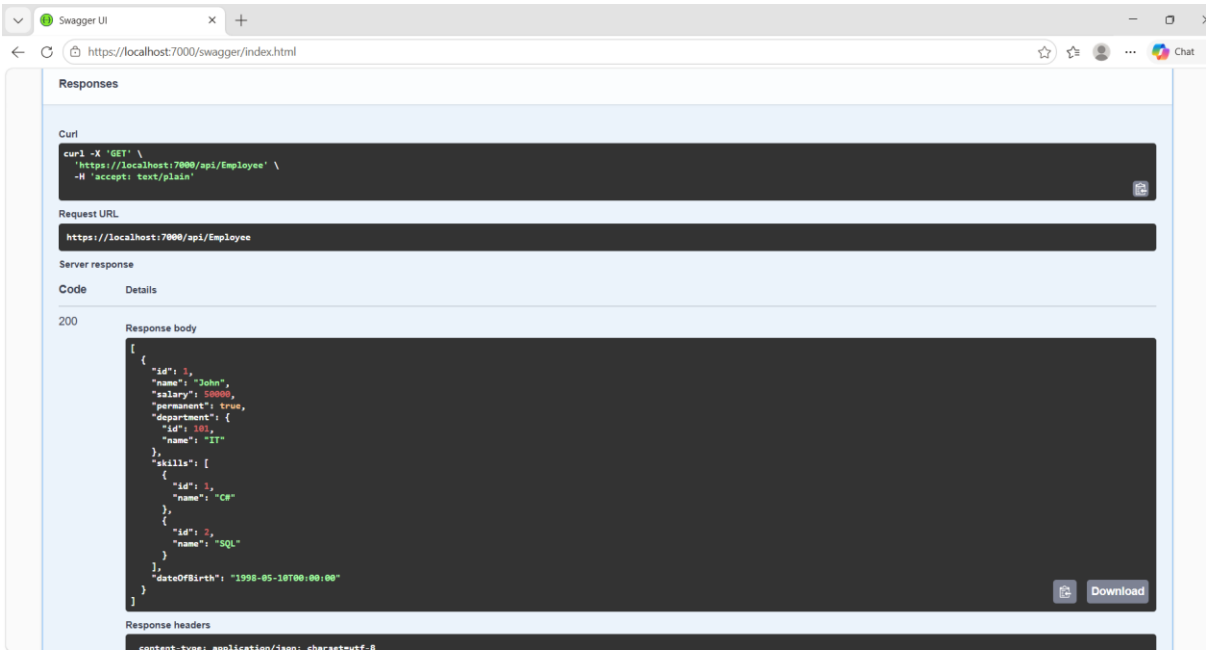
Result

Successfully created an ASP.NET Core Web API using a custom model class, implemented CRUD operations, used the FromBody and AllowAnonymous attributes, created a custom authorization filter using ActionFilterAttribute, implemented a custom exception filter using IExceptionFilter, and tested the APIs successfully using Swagger.

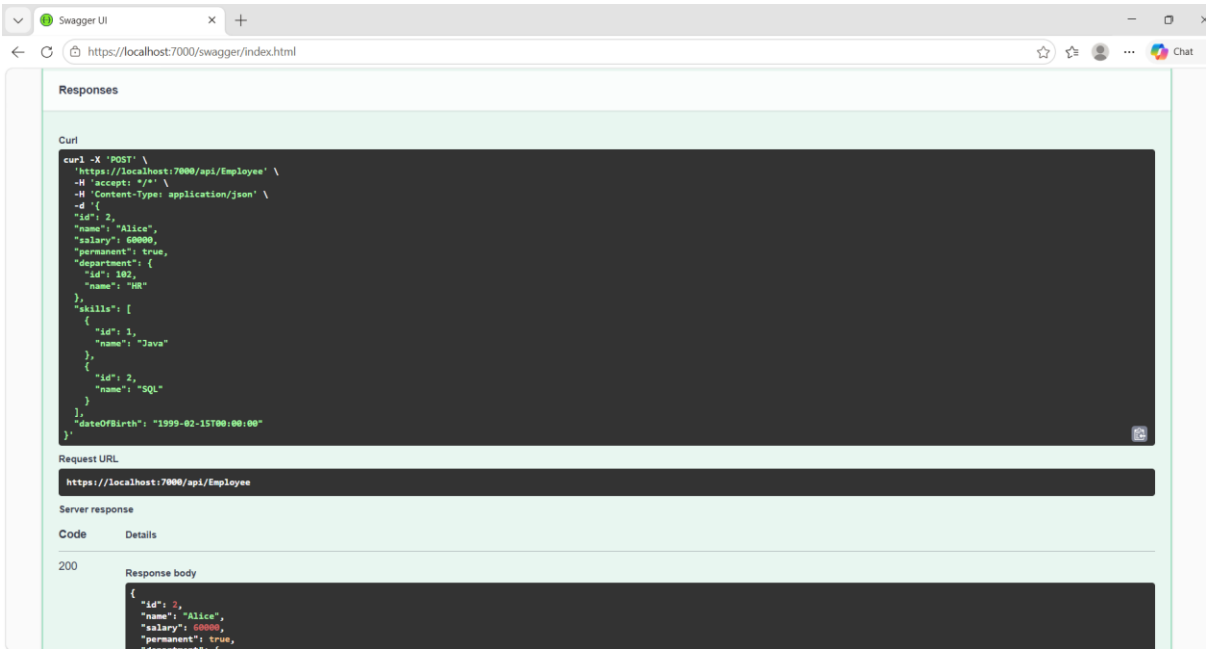
Outputs:



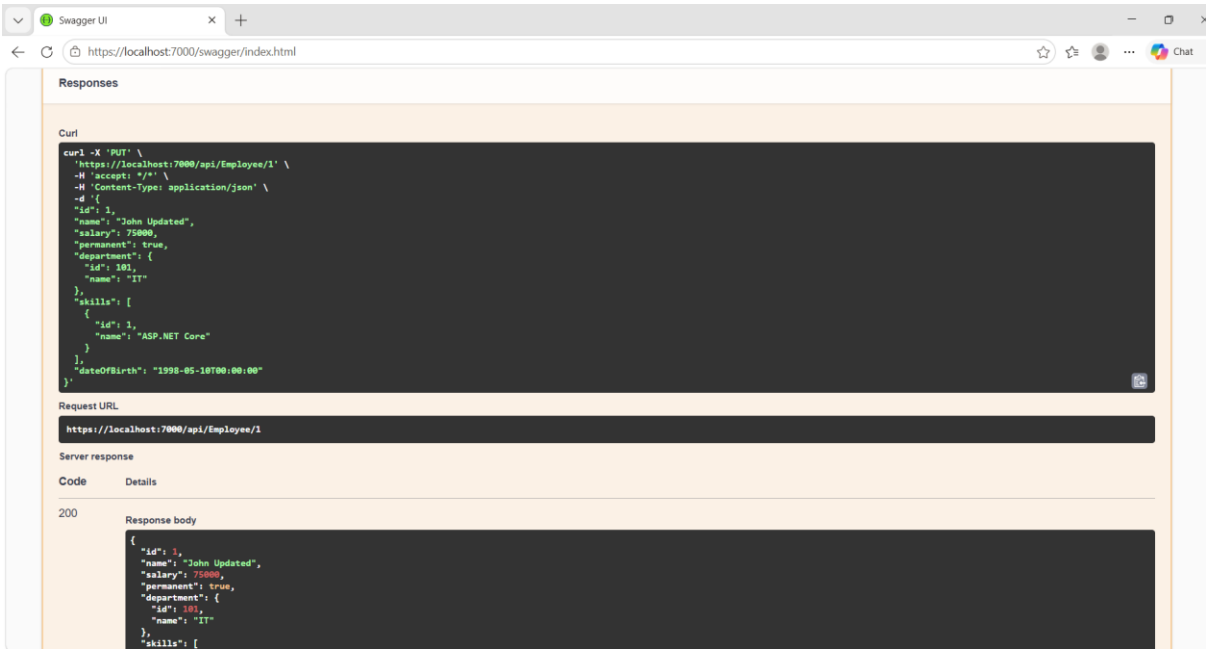
GET METHOD:



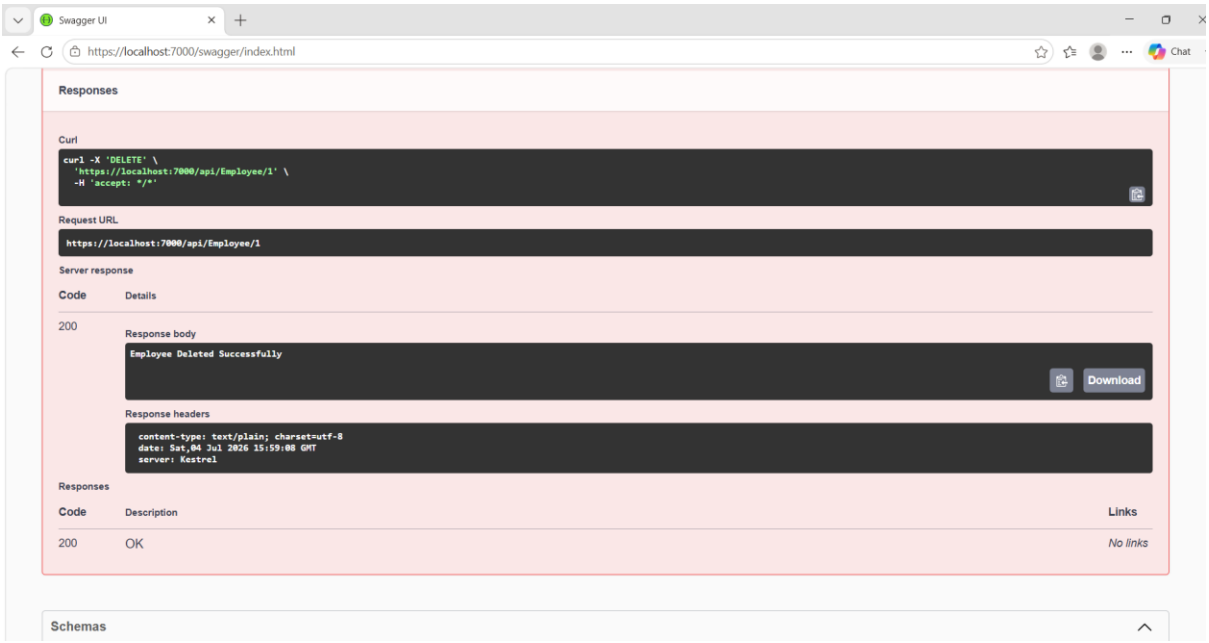
POST METHOD:



PUT METHOD:



DELETE METHOD:


Swagger UI
https://localhost:7000/swagger/index.html
Chat
Responses
Curl
curl -X 'DELETE' \
'https://localhost:7000/api/Employee/1' \
-H 'accept: */*'
Request URL
https://localhost:7000/api/Employee/1
Server response
Code
Details
200
Response body
Employee Deleted Successfully
Download
Response headers
content-type: text/plain; charset=utf-8
date: Sat,04 Jul 2026 15:59:08 GMT
server: Kestrel
Responses
Code
Description
Links
200
OK
No links
Schemas